Aula 20/09 Professor Thiago

PROCEDURE (também chamada stored procedure) é uma subrotina que fica armazenada no banco de dados.

Procedure para tratamento de erro

Criar um banco de dados.

```
1 create database aula2009;
```

Criar uma Tabela.

```
1 CREATE TABLE Alunos (
2     id INT AUTO_INCREMENT PRIMARY KEY,
3     nome VARCHAR(250),
4     curso VARCHAR(250),
5     salario DOUBLE,
6     idade INT
7 );
```

Aula 20/09 Professor Thiago

Tratamento de Erros em Procedimentos Armazenados

Exemplo como criar um procedimento armazenado que insere um aluno e trata erros que possam ocorrer: código cria um procedimento armazenado que tenta inserir um novo aluno na tabela `Alunos`. Se ocorrer um erro durante a inserção, ele captura esse erro e retorna uma mensagem apropriada. Se a inserção for bem-sucedida, uma mensagem de sucesso é retornada. Isso permite um tratamento de erros mais organizado e eficaz em suas operações de banco de dados.

```
1  DELIMITER //
2
3  CREATE PROCEDURE InserirAluno(
4      IN p_nome VARCHAR(250),
5      IN p_curso VARCHAR(250),
6      IN p_salario DOUBLE,
7      IN p_idade INT
8  )
9  BEGIN
10     DECLARE EXIT HANDLER FOR SQLEXCEPTION
11     BEGIN
12         -- Tratamento de erro
13         SELECT 'Erro ao inserir aluno' AS mensagem;
14     END;
15
16     -- Inserção do aluno
17     INSERT INTO Alunos (nome, curso, salario, idade) VALUES (p_nome, p_cur
   p_salario, p_idade);
18
19     SELECT 'Aluno inserido com sucesso' AS mensagem;
20 END //
21
22 DELIMITER ;
```

```
1 DELIMITER //
2
3 CREATE PROCEDURE InserirAluno(
4     IN p_nome VARCHAR(250),
5     IN p_curso VARCHAR(250),
6     IN p_salario DOUBLE,
7     IN p_idade INT
8 )
```

DELIMITER //

O comando `DELIMITER //` altera o delimitador padrão de instruções SQL (que é `;`) para `//`. Isso é necessário porque o procedimento armazenado contém várias instruções SQL, incluindo a instrução `END;`, que também usa `;`. Ao mudar o delimitador para `//`, o MySQL sabe que você está definindo um bloco de código que termina com `//`, permitindo que ele processe o procedimento como um todo.

CREATE PROCEDURE InserirAluno(...)

Essa linha inicia a definição de um procedimento armazenado chamado `InserirAluno`. Os parâmetros que seguem `IN` indicam que são entradas para o procedimento:

- `p_nome VARCHAR(250)`: Nome do aluno.
- `p_curso VARCHAR(250)`: Curso do aluno.
- `p_salario DOUBLE`: Salário do aluno.
- `p_idade INT`: Idade do aluno.

```
9 BEGIN
10     DECLARE EXIT HANDLER FOR SQLEXCEPTION
11     BEGIN
12         -- Tratamento de erro
13         SELECT 'Erro ao inserir aluno' AS mensagem;
14     END;
```

BEGIN ... END

As instruções entre `BEGIN` e `END` definem o corpo do procedimento. Tudo que estiver dentro desse bloco será executado quando o procedimento for chamado.

DECLARE EXIT HANDLER FOR SQLEXCEPTION

Este trecho declara um manipulador de exceção que será ativado se ocorrer um erro SQL durante a execução do procedimento. O manipulador é definido para "sair" (EXIT) e será ativado no caso de um `SQLEXCEPTION`, ou seja, qualquer erro SQL.

Dentro do manipulador, você pode definir o que deve acontecer em caso de erro. Neste caso, ele retorna uma mensagem:

```
    -- Tratamento de erro
    SELECT 'Erro ao inserir aluno' AS mensagem;
END
```

```
16     -- Inserção do aluno
17     INSERT INTO Alunos (nome, curso, salario, idade) VALUES (p_nome, p_curs
p_salario, p_idade);
18
```

Essa linha insere um novo registro na tabela `Alunos` com os valores recebidos pelos parâmetros. Se a inserção for bem-sucedida, a execução do código continuará até o final.

```
18
19     SELECT 'Aluno inserido com sucesso' AS mensagem;
20 END //
```

SELECT 'Aluno inserido com sucesso' AS mensagem

Após a inserção, essa linha retornará uma mensagem indicando que o aluno foi inserido com sucesso.

```
21
22 DELIMITER ;
```

Finalmente, o `END //` finaliza a definição do procedimento e `DELIMITER ;` redefine o delimitador de volta para o padrão (`;`).

Usando o Procedimento

Para usar o procedimento e tratar possíveis erros:

```
CALL InserirAluno('Ana Costa', 'Arquitetura', 4000.00, 23);
```

CALL:

Este comando é usado para chamar ou invocar um procedimento armazenado em MySQL. Ele permite que você execute a lógica definida dentro do procedimento que foi criado anteriormente.